

DevFlow — Full Project Specification

Flagship Full-Stack Developer Collaboration & Project Management SaaS

Core idea: DevFlow is a platform where software teams manage projects, tasks, issues and sprints, collaborate in real time, connect GitHub repositories, and understand engineering activity from one place.

Goal: Build a serious portfolio project that teaches full-stack engineering, backend architecture, PostgreSQL/Prisma, Redis, real-time systems, integrations, testing and later Docker/CI-CD/AI.

1. Technology Stack

Layer	Technology
Frontend	React + TypeScript + Tailwind CSS
Backend	Node.js + Express + TypeScript
Database	PostgreSQL + Prisma ORM
Auth / Storage	Supabase Auth + Supabase Storage
Caching / Rate limiting	Redis
Validation	Zod
Testing	Vitest/Jest + Supertest
Later	Docker + CI/CD + AWS + AI

2. Users & Roles

- **Admin:** manages users, workspaces, projects, roles and settings.
- **Project Manager:** manages members, tasks, issues, sprints and analytics.
- **Developer:** works on tasks/issues, comments, uploads files and connects GitHub.

3. Workspace & Project System

- Workspaces represent teams and contain members and projects.
- Projects contain overview, members, tasks, issues, sprints, activity, GitHub and analytics.
- Permissions are scoped to the relevant workspace/project.

4. Task Management & Kanban

- Tasks contain title, description, status, priority, assignee, labels, due date, sprint, comments, attachments and activity history.
- Kanban flow: TODO → IN PROGRESS → REVIEW → DONE.
- Dragging a task updates the backend; later, other users receive the change in real time.

5. Issue Tracking

- Dedicated issues with title, description, priority, status, reporter, assignee, labels, comments, attachments, related tasks and activity history.

6. Sprint Management

- Create sprints with dates and assign tasks.
- Track completed, in-progress and remaining work, velocity and burndown-style progress.

7. Collaboration

- Comments on tasks/issues.
- Activity feed for assignments, status changes, issue creation, PR events and other important actions.
- Notifications for assignments, mentions, comments, PRs and deadlines.

8. File Attachments

- Use Supabase Storage for screenshots, PDFs, design files and logs.
- Store file metadata/ownership in PostgreSQL and file bytes in object storage.
- Validate file type, size and authorization.

9. Redis

- Cache frequently accessed data.
- Implement API rate limiting.
- Use short-lived data where appropriate.
- Learn cache-aside, TTL, cache misses and cache invalidation.

10. Real-Time Collaboration

- Use WebSockets/Socket.IO for task updates, comments, notifications and selected presence/typing features.
- Design events so the system can later work across multiple backend instances.

11. GitHub Integration

- Connect GitHub with OAuth.
- Display repositories, commits, branches and pull requests.
- Link DevFlow issues to GitHub PRs.
- Use webhooks and learn retries, signature verification and idempotency.

12. Engineering Analytics

- Project dashboard: tasks, issues, sprint progress, PRs and commits.
- Later metrics: issue resolution time, sprint velocity and project progress.
- Use analytics as project signals rather than simplistic developer productivity scores.

13. Backend Architecture

- Request → Route → Middleware → Controller → Service → Repository/Prisma → PostgreSQL.
- Keep controllers thin and business logic in services.
- Use centralized error handling, validation, pagination, filtering and consistent API responses.

14. Database Model

- Core entities: User, Workspace, WorkspaceMember, Project, ProjectMember, Task, TaskLabel, Issue, Comment, Attachment, Sprint, Notification, Activity, GitHubAccount, GitHubRepository, PullRequest and WebhookEvent.
- Model relationships explicitly with Prisma/PostgreSQL and add indexes based on query patterns.

15. Security

- Supabase authentication plus RBAC/resource-level authorization.

- Input validation, safe secrets, CORS/security configuration, rate limiting, file validation and verified GitHub webhooks.
- Use Prisma safely and avoid exposing sensitive internal errors.

16. Testing

- Unit tests for important business logic.
- API/integration tests for authentication, authorization and core endpoints.
- Edge-case/failure testing and later load/concurrency tests.

17. Docker — Later

- Do not begin with Docker. Learn Docker first, then return to DevFlow.
- Eventually use Docker/Docker Compose for reproducible local services and containerized components.
- Add health checks and environment-specific configuration.

18. CI/CD & Deployment — Later

- GitHub → CI checks → tests/build → Docker → deployment.
- Add secrets, health checks, logging and basic monitoring.

19. AI — Later After AI Learning

- AI issue breakdown into implementation tasks.
- AI PR summarization.
- Repository Q&A; using connected repository context.
- Sprint analysis and release-note generation.
- Put AI behind a clean service boundary with usage limits, logging, cost controls and output evaluation.

20. Overall Architecture

[illegible]

21. Recommended Build Order

1. Requirements, wireframes and architecture.
2. Project setup and conventions.
3. Supabase authentication + RBAC.
4. Workspaces, members and projects.
5. Tasks + Kanban.
6. Issues + comments + attachments.
7. Sprints + analytics.
8. Redis caching + rate limiting.
9. Real-time collaboration + notifications.
10. GitHub OAuth + repository/PR integration + webhooks.
11. Testing, validation, error handling, security and performance.
12. Docker after learning Docker.
13. CI/CD and deployment.
14. AI features after the planned AI-learning phase.

22. What This Project Should Prove

- React/TypeScript frontend development.
- REST API and backend architecture.
- PostgreSQL relational modeling with Prisma.
- Redis caching and rate limiting.
- Real-time events, webhooks, retries and idempotency.
- OAuth and third-party API integration.
- Testing, validation, security and documentation.
- Docker/CI-CD/deployment after learning those concepts.

- Meaningful AI integration later, rather than a superficial chatbot.

Success criterion: AI agents may accelerate implementation, but you should understand and be able to explain the architecture, code, database design, security, trade-offs and failure modes yourself.